

TÀI LIỆU KỸ THUẬT

KIẾN TRÚC VÀ CƠ CHẾ HOẠT ĐỘNG CỦA BROWSER HELPER OBJECT

COM • Internet Explorer • Connection Point • Event Sink • Reverse Engineering

Mục tiêu của tài liệu

Giải thích BHO từ nền tảng COM đến quá trình reverse một DLL đã strip: nhận diện CLSID/IID/DIID, dựng lại SetSite, ánh xạ vtable, lần theo IWebBrowser2, giải mã IDispatch::Invoke và đánh giá hành vi malware.

Phiên bản 1.0 • 2026

Dành cho người học Windows Reverse Engineering và Malware Analysis

Lời nói đầu và cách sử dụng tài liệu

Browser Helper Object là một chủ đề rất phù hợp để học reverse engineering trên Windows vì nó buộc người phân tích phải nối nhiều lớp kiến thức: PE/DLL, Registry, COM, con trỏ đối tượng, vtable, callback, kiểu VARIANT và mô hình sự kiện của Internet Explorer. Nếu chỉ tìm chuỗi hoặc xem Imports, ta sẽ thấy rất ít. Hành vi quan trọng thường xuất hiện dưới dạng lời gọi gián tiếp qua vtable, ví dụ `call dword ptr [eax+2Ch]`.

Tài liệu được tổ chức theo hai góc nhìn song song. Góc nhìn hệ thống giải thích Windows và Internet Explorer tạo, gắn và gọi BHO như thế nào. Góc nhìn reverse chỉ ra dấu hiệu cụ thể trong IDA/Ghidra: tham số nào là IID, field nào giữ interface pointer, offset nào là method, HRESULT được kiểm tra ra sao và nơi nào là callback thực sự chứa hành vi.

Nguyên tắc đọc

Không học thuộc toàn bộ vtable. Hãy học chuỗi suy luận: call site → prototype → IID → interface type → vtable layout → method name → semantics. Phụ lục cuối tài liệu được thiết kế như cheat sheet để tra cứu nhanh.

Phạm vi

- Tập trung vào BHO cổ điển chạy trong Internet Explorer trên Windows, đặc biệt mô hình COM in-process.
- Bao gồm cả BHO hợp pháp lẫn cách adware/spyware lạm dụng cùng kiến trúc.
- Mã và pseudocode phục vụ phân tích phòng thủ, reverse engineering và nhận diện hành vi.
- Không xem CLSID hay một API đơn lẻ là bằng chứng tuyệt đối; kết luận phải dựa trên registry, call graph, GUID, vtable và dữ liệu runtime.

Quy ước

Ký hiệu	Ý nghĩa
this + 0x10	Field nằm tại offset 0x10 trong object C++/COM đang phân tích.
vtable + 0x2C	Entry method tại byte offset 0x2C trong vtable 32-bit.
SUCCEEDED(hr)	HRESULT có bit dấu bằng 0; decompiler thường hiện <code>hr >= 0</code> .
[in] / [out]	Hướng truyền dữ liệu theo prototype COM/WinAPI.
source / sink	Đối tượng phát sự kiện / đối tượng nhận callback.

Mục lục

Lời nói đầu và cách sử dụng tài liệu	2
Phạm vi	2
Quy ước	2
Mục lục	3
PHẦN 1. TỔNG QUAN VỀ BROWSER HELPER OBJECT	5
1.1 BHO là gì?	5
1.2 Ba đặc tính quyết định rủi ro	5
1.3 BHO trong mô hình tiến trình IE	5
1.4 Dấu hiệu BHO trong phân tích tĩnh	5
PHẦN 2. KIẾN TRÚC COM NỀN TẢNG CỦA BHO	7
2.1 COM object, interface và identity	7
2.2 IUnknown và luật nền tảng	7
2.3 IClassFactory: object được tạo như thế nào	7
2.4 IObjectWithSite: cầu nối giữa host và extension	8
2.5 IWebBrowser2 và cây kế thừa	8
2.6 Cách nhận diện interface trong binary	8
PHẦN 3. ĐĂNG KÝ VÀ TẢI NẠP	9
3.1 Hai nhánh Registry có hai vai trò khác nhau	9
3.2 Self-registration	9
3.3 Luồng tải nạp đầy đủ	9
3.4 DllMain không phải entry point hành vi	9
3.5 Static triage checklist	10
PHẦN 4. VÒNG ĐỜI VÀ IObjectWithSite	11
4.1 Chuỗi tạo object	11
4.2 SetSite khi khởi tạo	11
4.3 SetSite khi teardown	11
4.4 Dựng lại object layout từ offset	11
4.5 Case study: nhận diện IWebBrowser2 từ IID	12
4.6 Phân tích hàm đăng ký event điển hình	12
PHẦN 5. CONNECTION POINT VÀ EVENT SINK	13
5.1 Mô hình source - sink	13
5.2 IConnectionPointContainer	13
5.3 IConnectionPoint	13
5.4 Advise và cookie	13
5.5 Event sink qua IDispatch	14

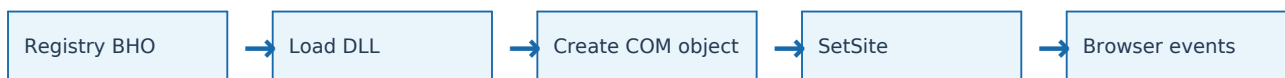
5.6 IDispatch::Invoke trong decompiler	14
5.7 Tham số đảo ngược trong DISPPARAMS	14
PHẦN 6. TRUY XUẤT VÀ CAN THIỆP DỮ LIỆU TRÌNH DUYỆT	15
6.1 DWebBrowserEvents2 quan trọng	15
6.2 BeforeNavigate2 và dữ liệu cleartext	15
6.3 VARIANT: đọc tag trước, dữ liệu sau	15
6.4 DOM access qua get_Document	15
6.5 Điều khiển trình duyệt: Navigate và put_Visible	16
6.6 Network hooking: phân biệt các lớp	16
6.7 Quy trình reverse hành vi từ Invoke	16
PHẦN 7. BẢO MẬT, PHÒNG VỆ VÀ SỰ THOÁI TRÀO	17
7.1 Vì sao BHO phù hợp với adware/spyware	17
7.2 Man-in-the-browser	17
7.3 Protected Mode và integrity level	17
7.4 32-bit và 64-bit	17
7.5 Dấu hiệu giám sát và hunting	17
7.6 COM hijacking và CLSID reuse	17
7.7 Vì sao BHO suy tàn	18
PHẦN 8. PHỤ LỤC KỸ THUẬT VÀ CHEAT SHEET	19
8.1 GUID chuẩn thường gặp	19
8.2 GUID endian conversion	19
8.3 IObjectWithSite vtable x86	19
8.4 IConnectionPointContainer vtable x86	20
8.5 IConnectionPoint vtable x86	20
8.6 IDispatch vtable x86	20
8.7 IWebBrowser2 offsets quan trọng x86	21
8.8 DISPIDs tiêu biểu	21
PHẦN 9. QUY TRÌNH REVERSE BHO THỰC CHIẾN	22
9.1 Mục tiêu đầu ra	22
9.2 Workflow trong IDA/Ghidra	22
9.3 Mẫu phân tích lời gọi gián tiếp	22
9.4 Debug checklist	23
9.5 Các bẫy thường gặp	23
9.6 Case walkthrough rút gọn	23
THUẬT NGỮ	24
TÀI LIỆU THAM KHẢO	24

PHẦN 1. TỔNG QUAN VỀ BROWSER HELPER OBJECT

1.1 BHO là gì?

Browser Helper Object (BHO) là một COM component dạng DLL được Internet Explorer hoặc Windows Explorer nạp vào chính tiến trình của chúng. Vì là in-process server, code của BHO thực thi trong cùng không gian địa chỉ với host. BHO không phải một chương trình độc lập và thường không có cửa sổ riêng; nó tồn tại như một đối tượng được host tạo thông qua COM.

Mục tiêu ban đầu của kiến trúc là mở rộng trình duyệt mà không sửa mã nguồn IE: thanh công cụ, download manager, trợ năng, dịch thuật, quản lý mật khẩu và lọc nội dung. Chính quyền truy cập sâu đó cũng khiến BHO trở thành nền tảng hấp dẫn cho adware và spyware.



BHO không đồng nghĩa malware

BHO là cơ chế mở rộng. Tính độc hại nằm ở hành vi: thay đổi trang, chèn quảng cáo, lấy form data, điều hướng sang URL khác, tải payload hoặc duy trì persistence. Khi phân tích, hãy mô tả capability trước khi gán nhãn.

1.2 Ba đặc tính quyết định rủi ro

Đặc tính	Hệ quả kỹ thuật	Góc nhìn phân tích
In-process	DLL chạy bên trong iexplore.exe/explorer.exe.	Crash của BHO có thể làm host crash; network/file activity mang danh host.
COM object	Hành vi đi qua interface pointer và vtable.	Imports thường không chứa tên các COM method; phải dựng kiểu.
Event driven	Phần lớn logic chỉ chạy khi IE phát sự kiện.	Entry point thật có thể là IDispatch::Invoke thay vì DllMain/SetSite.

1.3 BHO trong mô hình tiến trình IE

Trên các phiên bản IE cũ, một hoặc nhiều cửa sổ/tab có thể dùng các process khác nhau tùy phiên bản và cấu hình. DLL BHO được map vào từng process đủ điều kiện. Điều này giải thích vì sao cùng BHO có thể có nhiều instance, nhiều reference count và nhiều connection cookie đồng thời.

Khi EPM/Protected Mode được bật, integrity level, bitness và chính sách add-on ảnh hưởng việc DLL có được nạp hay không. Một BHO 32-bit không thể được nạp trực tiếp vào process IE 64-bit và ngược lại.

1.4 Dấu hiệu BHO trong phân tích tĩnh

- Exports thường gặp: DllGetClassObject, DllCanUnloadNow, DllRegisterServer, DllUnregisterServer.
- Strings registry: Browser Helper Objects, CLSID, InprocServer32, ThreadingModel, NoExplorer.
- GUID: IID_IObjectWithSite, IID_IWebBrowser2, IID_IConnectionPointContainer, DIID_DWebBrowserEvents2.
- Lời gọi COM gián tiếp qua vtable; nhiều method quan trọng không xuất hiện trong Import Address Table.

- Callback `IDispatch::Invoke` xử lý `DISPID` thay vì một hàm có tên `BeforeNavigate2` hay `DocumentComplete`.

PHẦN 2. KIẾN TRÚC COM NỀN TẢNG CỦA BHO

2.1 COM object, interface và identity

COM tách khái niệm object khỏi interface. Một object có thể cung cấp nhiều interface; client không truy cập field nội bộ mà chỉ giữ con trỏ interface. Mỗi interface được nhận diện bằng IID và có một vtable cố định. CLSID nhận diện class có khả năng tạo object, không nhận diện interface.

Identifier	Trả lời câu hỏi	Ví dụ trong BHO
CLSID	Tạo class/object nào?	CLSID của BHO được ghi dưới InprocServer32 và Browser Helper Objects.
IID	Muốn con trỏ interface nào?	IID_IWebBrowser2, IID_IObjectWithSite.
DIID	Dispatch/event interface nào?	DIID_DWebBrowserEvents2.
DISPID	Member/event nào trong IDispatch?	DISPID_BEFORENAVIGATE2 = 250.

Sai lầm phổ biến

Tra CLSID rồi kết luận interface là IWebBrowser2 là sai tâng. CLSID chỉ nhận diện class. Interface phải được chứng minh bằng IID trong QueryInterface, prototype trả về, type library hoặc sự khớp vtable.

2.2 IUnknown và luật nền tảng

```
interface IUnknown {
HRESULT QueryInterface(REFIID riid, void **ppv); // slot 0
ULONG AddRef(); // slot 1
ULONG Release(); // slot 2
};
```

Ba method đầu tiên xuất hiện ở đầu mọi COM vtable. Trong binary x86, chúng lần lượt ở offset 0x00, 0x04 và 0x08. Đây là anchor quan trọng nhất khi reverse. Một indirect call ở slot 0 với hai tham số sau this - một con trỏ 16 byte và một địa chỉ output - gần như chắc chắn là QueryInterface.

QueryInterface không tạo object mới. Nó yêu cầu cùng object trả về một view/interface khác. Nếu thành công, output nhận interface pointer và reference count thường được tăng. Client phải Release khi không dùng nữa.

HRESULT trong decompiler

```
HRESULT hr = obj->QueryInterface(iid, &out);
if (SUCCEEDED(hr)) { ... }

// Decompiler thường biếm di n:
if (result >= 0) { ... }
```

HRESULT thành công có bit cao bằng 0. Vì vậy kiểm tra signed hr >= 0 là dấu hiệu mạnh của một call COM/WinAPI trả HRESULT.

2.3 IClassFactory: object được tạo như thế nào

Khi COM runtime cần một in-process class, nó nạp DLL và gọi DllGetClassObject với CLSID cùng IID_IClassFactory. DLL trả về IClassFactory; runtime sau đó gọi CreateInstance để tạo instance. LockServer kiểm soát việc DLL có thể unload khi không còn object hay không.

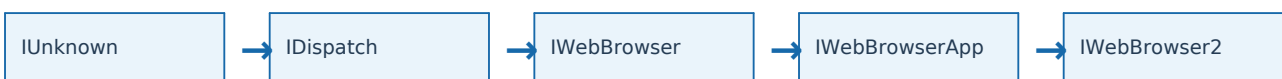
```
DllGetObject(rclsid, IID_IClassFactory, &factory);
factory->CreateInstance(NULL, requestedIID, &object);
factory->Release();
```

2.4 IObjectWithSite: cầu nối giữa host và extension

```
interface IObjectWithSite : IUnknown {
HRESULT SetSite(IUnknown *pUnkSite); // +0x0C
HRESULT GetSite(REFIID riid, void **ppvSite); // +0x10
};
```

Host gọi SetSite để trao cho extension một object site. pUnkSite được khai báo IUnknown vì host không muốn buộc extension vào một interface duy nhất. BHO dùng QueryInterface trên pUnkSite để xin IWebBrowser2 hoặc interface liên quan. Khi pUnkSite bằng NULL, BHO phải tháo event subscription và Release các interface pointer đã giữ.

2.5 IWebBrowser2 và cây kế thừa



IWebBrowser2 đại diện cho browser automation object. Vì kế thừa dài, vtable của nó chứa toàn bộ method của các interface cha trước khi đến các method riêng. Đây là lý do Navigate nằm ở 0x2C và put_Visible nằm tận 0xA4 trong x86.

Offset x86	Method	Ý nghĩa
0x00	QueryInterface	Xin một interface khác trên cùng object.
0x18	IDispatch::Invoke	Late binding/automation invocation.
0x2C	IWebBrowser::Navigate	Điều hướng trình duyệt đến URL/path.
0x48	get_Document	Lấy automation object của document hiện tại.
0xA0	get_Visible	Đọc trạng thái hiển thị.
0xA4	put_Visible	Hiện hoặc ẩn cửa sổ browser.

2.6 Cách nhận diện interface trong binary

Có bốn mức bằng chứng. Mức mạnh nhất là GUID truyền trực tiếp vào QueryInterface. Mức tiếp theo là prototype của method trả về một interface cụ thể. Mức thứ ba là type library/symbol. Cuối cùng mới là suy luận từ offset, số tham số và cách dùng kết quả.

```
// Pseudocode decompiler
>(*site + 0)(site, &riid, &browser);

// riid = {D30C1661-CDAF-11D0-8A3E-00C04FC9E26E}
// => IID_IWebBrowser2
// => browser có kiểu IWebBrowser2 *
```

Quy tắc vàng khi reverse COM

IID xác định kiểu interface. Vtable offset xác định method. Prototype và số tham số dùng để kiểm chứng. Không bắt đầu bằng việc đoán hành vi từ một offset cô lập.

PHẦN 3. ĐĂNG KÝ VÀ TẢI NẠP

3.1 Hai nhánh Registry có hai vai trò khác nhau

```
HKCR\CLSID\{BHO-CLSID}\InprocServer32
(Default) = C:\Path\Bho.dll
ThreadingModel = Apartment
```

```
HKLM\Software\Microsoft\Windows\CurrentVersion\Explorer\
Browser Helper Objects\{BHO-CLSID}
NoExplorer = 1 // tùy chọn
```

Nhánh CLSID nói cho COM biết DLL nào hiện thực class. Nhánh Browser Helper Objects nói cho IE/Explorer biết class nào cần được khởi tạo như extension. Chỉ có một nhánh thường chưa đủ để tạo hành vi BHO hoàn chỉnh.

Artifact	Câu hỏi điều tra	Ý nghĩa
InprocServer32	DLL nằm ở đâu?	Đường dẫn payload được host LoadLibrary.
ThreadingModel	Object kỳ vọng apartment nào?	BHO thường dùng Apartment do IE UI/COM automation.
Browser Helper Objects	Class có được host enumerate không?	Điểm persistence/load trigger.
NoExplorer	Có nạp trong explorer.exe không?	Giảm phạm vi xuống Internet Explorer.

3.2 Self-registration

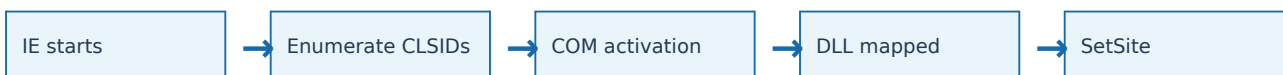
regsvr32 không phân tích DLL để tự biết registry cần tạo. Nó nạp DLL rồi gọi export DllRegisterServer. Chính code của DLL tạo các key/value cần thiết. DllUnregisterServer thực hiện chiều ngược lại.



Góc nhìn malware analysis

- Tìm command line regsvr32 /s trong strings, process tree hoặc script dropper.
- Theo dõi RegCreateKeyEx/RegSetValueEx để xác định CLSID, InprocServer32 và BHO key.
- Kiểm tra bitness: regsvr32 trong System32 là 64-bit; SysWOW64 là 32-bit.
- Nếu DLL export registration nhưng không thấy key, code có thể kiểm tra quyền hoặc môi trường trước khi ghi.

3.3 Luồng tải nạp đầy đủ



Host enumerate CLSID trong nhánh BHO, kích hoạt COM class, tạo instance và gán site. DllMain chỉ nên thực hiện khởi tạo tối thiểu vì đang chạy dưới loader lock. Logic phức tạp thường xuất hiện sau CreateInstance hoặc SetSite.

3.4 DllMain không phải entry point hành vi

Trong BHO, DllMain có thể chỉ lưu HMODULE và gọi DisableThreadLibraryCalls. Nếu analyst dừng tại đây và không tìm SetSite/Invoke, phần lớn hành vi sẽ bị bỏ qua. Entry point phân tích nên được hiểu là một tập: exports COM, SetSite, event callback và các worker được callback kích hoạt.

3.5 Static triage checklist

Bước	Thao tác	Kết quả mong đợi
1	Liệt kê exports.	Nhận diện COM server/self-registration.
2	Tìm strings registry/CLSID.	Xác định class và load trigger.
3	Định dạng các GUID 16 byte.	Nhận diện IID/DIID.
4	Tìm vtable và constructors.	Dựng object layout.
5	Lần SetSite và Invoke.	Đến logic browser/event.
6	Đối chiếu dynamic trace.	Xác nhận URL, file, registry, callback.

PHẦN 4. VÒNG ĐỜI VÀ IObjectWithSite

4.1 Chuỗi tạo object

```
1. DllMain(DLL_PROCESS_ATTACH)
2. DllGetObject(CLSID_BHO, IID_IClassFactory, &factory)
3. factory->CreateInstance(NULL, IID_IObjectWithSite, &bho)
4. bho->SetSite(pUnkSite)
```

Trong thực tế, IID yêu cầu tại CreateInstance có thể là IUnknown rồi host QueryInterface tiếp. Điểm ổn định cần nhớ là object BHO phải cung cấp IObjectWithSite để nhận site pointer.

4.2 SetSite khi khởi tạo

```
HRESULT BHO::SetSite(IUnknown *site) {
if (site != NULL) {
site->QueryInterface(IID_IWebBrowser2, (void**)&browser);
browser->QueryInterface(IID_IConnectionPointContainer, (void**)&cpc);
cpc->FindConnectionPoint(DIID_DWebBrowserEvents2, &cp);
cp->Advise(static_cast<IDispatch*>(this), &cookie);
}
return S_OK;
}
```

Không phải implementation nào cũng đi đúng chuỗi trên. Một số site object hỗ trợ trực tiếp IServiceProvider rồi QueryService để lấy browser; một số dùng ATL helper. Vì vậy interface phải được nhận diện từ GUID và prototype thực tế.

4.3 SetSite khi teardown

```
if (site == NULL) {
if (cp && cookie) cp->Unadvise(cookie);
cookie = 0;
SAFE_RELEASE(cp);
SAFE_RELEASE(browser);
SAFE_RELEASE(oldSite);
}
```

Nếu thiếu Unadvise, source có thể tiếp tục callback tới sink đã bị giải phóng, gây use-after-free hoặc crash. Nếu thiếu Release, IE giữ object/DLL lâu hơn và rò reference. Trong malware, cleanup có thể cố ý sơ sài vì persistence/ngắn hạn quan trọng hơn độ ổn định.

4.4 Dựng lại object layout từ offset

```
// Decompiler
*(_DWORD*)(this + 0x10) = browser;
*(_DWORD*)(this + 0x14) = connectionPoint;
*(_DWORD*)(this + 0x18) = cookie;
```

Offset	Kiểu đề xuất	Bằng chứng
+0x00	vtable pointer	Object COM/C++ thông thường.
+0x10	IWebBrowser2 *	Output của QueryInterface(IID_IWebBrowser2).
+0x14	IConnectionPoint *	Output của FindConnectionPoint.
+0x18	DWORD cookie	Output thứ hai của Advise; input của Unadvise.

Sau khi có layout, hãy tạo struct trong IDA/Ghidra và áp vào this pointer. Pseudocode sẽ chuyển từ phép cộng số học sang field có tên, giúp các hàm khác tự lộ ý nghĩa.

4.5 Case study: nhận diện IWebBrowser2 từ IID

```
riid IID <0D30C1661h, 0CDAFh, 11D0h,
<8Ah, 3Eh, 00h, 0C0h, 4Fh, 0C9h, 0E2h, 6Eh>>
```

Cấu trúc trên biểu diễn GUID D30C1661-CD AF-11D0-8A3E-00C04FC9E26E, tức IID_IWebBrowser2. Số 0 đứng trước D30C1661 chỉ là cú pháp assembler cho hằng hex bắt đầu bằng chữ. Output pointer của QueryInterface từ thời điểm này phải được type là IWebBrowser2*.

Không phải tự đoán method

Sau khi biết output là IWebBrowser2*, hãy tra đúng vtable và ánh xạ offset. Ví dụ x86: +0x2C = Navigate, +0xA4 = put_Visible. Việc nhận diện là tất định nếu IID và offset đúng.

4.6 Phân tích hàm đăng ký event điển hình

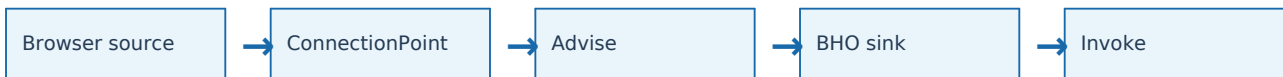
```
if (this->browser != NULL) {
hr = this->browser->QueryInterface(IID_IConnectionPointContainer, &cpc);
if (SUCCEEDED(hr)) {
hr = cpc->FindConnectionPoint(DIID_DWebBrowserEvents2, &this->cp);
if (SUCCEEDED(hr))
return this->cp->Advise(eventSink, &this->cookie);
cpc->Release();
}
}
```

Ba call có thể xuất hiện trong decompiler dưới dạng triple dereference. Hãy bám theo loại con trỏ: browser -> QueryInterface output cpc -> FindConnectionPoint output cp -> Advise output cookie. Đây là kỹ thuật type propagation, hiệu quả hơn đọc từng instruction rời rạc.

PHẦN 5. CONNECTION POINT VÀ EVENT SINK

5.1 Mô hình source - sink

COM connection point là cơ chế publish/subscribe. Browser là source phát event; BHO là sink nhận event. IConnectionPointContainer thuộc source và chứa một hay nhiều IConnectionPoint, mỗi connection point tương ứng một outgoing interface.



5.2 IConnectionPointContainer

Slot	Offset x86	Method	Vai trò
0	0x00	QueryInterface	IUnknown.
1	0x04	AddRef	IUnknown.
2	0x08	Release	IUnknown.
3	0x0C	EnumConnectionPoints	Liệt kê outgoing interfaces.
4	0x10	FindConnectionPoint	Tìm IConnectionPoint theo IID/DIID.

```

HRESULT FindConnectionPoint(
    REFIID riid,
    IConnectionPoint **ppCP
);
  
```

Trong decompiler x86, tính cả this sẽ có ba đối số. Nếu call ở +0x10 nhận một con trỏ GUID và một output pointer, đó là bằng chứng rất mạnh mẽ cho FindConnectionPoint.

5.3 IConnectionPoint

Slot	Offset	Method	Vai trò
0-2	0x00-0x08	IUnknown	Identity và lifetime.
3	0x0C	GetConnectionInterface	Lấy IID outgoing interface.
4	0x10	GetConnectionPointContainer	Quay lại container.
5	0x14	Advise	Đăng ký sink, nhận cookie.
6	0x18	Unadvise	Hủy đăng ký bằng cookie.
7	0x1C	EnumConnections	Liệt kê sink đang đăng ký.

5.4 Advise và cookie

```

HRESULT Advise(
    IUnknown *pUnkSink,
    DWORD *pdwCookie
);
  
```

Advise không chỉ lưu một function pointer. Connection point QueryInterface sink để lấy outgoing interface mà nó yêu cầu. Với DWebBrowserEvents2, sink thường hiện thực IDispatch. Nếu thành công, cookie nhận token duy nhất dùng cho Unadvise.

5.5 Event sink qua IDispatch

Offset x86	IDispatch method	Trong sink BHO
0x00	QueryInterface	Phải trả IUnknown/IDispatch phù hợp.
0x04	AddRef	Giữ lifetime khi source kết nối.
0x08	Release	Giảm reference.
0x0C	GetTypeInfoCount	Có thể trả 0/E_NOTIMPL.
0x10	GetTypeInfo	Thường tối giản.
0x14	GetIDsOfNames	Có thể không cần cho event sink.
0x18	Invoke	Dispatcher chính cho browser events.

5.6 IDispatch::Invoke trong decompiler

```
HRESULT Invoke(
DISPID dispIdMember,
REFIID riid,
LCID lcid,
WORD wFlags,
DISPPARAMS *pDispParams,
VARIANT *pVarResult,
EXCEPINFO *pExcepInfo,
UINT *puArgErr
);
```

Tham số quan trọng nhất thường là dispIdMember và pDispParams. Compiler có thể khiến prototype bị mất; hãy nhận diện hàm có nhiều tham số, switch trên một integer nhỏ/0xFA và truy cập mảng VARIANT trong DISPPARAMS.

5.7 Tham số đảo ngược trong DISPPARAMS

Automation truyền positional arguments theo thứ tự phải sang trái. rgvarg[0] là tham số cuối cùng trong định nghĩa event. Đây là nguồn lỗi phổ biến khi analyst gán nhầm URL, headers, POST data và Cancel.

```
typedef struct tagDISPPARAMS {
VARIANTARG *rgvarg;
DISPID *rgdispidNamedArgs;
UINT cArgs;
UINT cNamedArgs;
} DISPPARAMS;
```

Checklist khi đọc Invoke

(1) Xác định DISPID; (2) tra prototype event; (3) đảo thứ tự tham số; (4) kiểm tra vt của từng VARIANT; (5) theo con trỏ BYREF nếu có; (6) chỉ gán Cancel khi đúng kiểu VT_BOOL|VT_BYREF.

PHẦN 6. TRUY XUẤT VÀ CAN THIỆP DỮ LIỆU TRÌNH DUYỆT

6.1 DWebBrowserEvents2 quan trọng

DISPID	Event	Thời điểm	Giá trị phân tích
250 / 0xFA	BeforeNavigate2	Trước điều hướng	URL, flags, target frame, POST data, headers, Cancel.
252 / 0xFC	NavigateComplete2	Điều hướng cơ bản hoàn tất	URL cuối và browser/frame object.
259 / 0x103	DocumentComplete	Document hoàn tất	Điểm thuận lợi để truy cập DOM.
251 / 0xFB	NewWindow2	Trước khi tạo window	Có thể thay browser object hoặc hủy.
273 / 0x111	NewWindow3	Biến thể mới	URL/context bổ sung.

6.2 BeforeNavigate2 và dữ liệu cleartext

BHO nằm bên trong browser nên quan sát dữ liệu ở lớp ứng dụng trước khi stack TLS mã hóa. Điều này không có nghĩa BHO phá TLS; nó đọc dữ liệu trước khi mã hóa hoặc sau khi giải mã trong chính endpoint. Vì vậy mô hình được gọi là man-in-the-browser, khác với man-in-the-middle trên mạng.

```
BeforeNavigate2(
    IDispatch *pDisp,
    VARIANT *URL,
    VARIANT *Flags,
    VARIANT *TargetFrameName,
    VARIANT *PostData,
    VARIANT *Headers,
    VARIANT_BOOL *Cancel
);
```

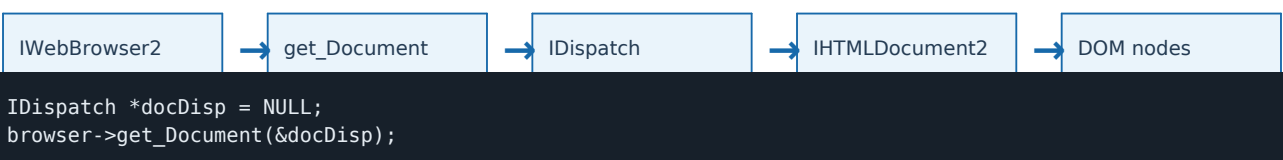
Trong DISPPARAMS, thứ tự thường bị đảo: Cancel ở rgvarg[0], Headers ở [1], PostData ở [2] và URL gần cuối. Phải kiểm tra vt trước khi dereference. PostData có thể là SAFEARRAY byte chứ không phải BSTR.

6.3 VARIANT: đọc tag trước, dữ liệu sau

VARTYPE	Ý nghĩa	Cách gộp trong BHO
VT_BSTR	Chuỗi BSTR	URL, headers, frame name.
VT_BOOL	VARIANT_BOOL	Visible, Cancel.
VT_DISPATCH	IDispatch *	Browser/frame/document automation object.
VT_ARRAY VT_UI1	SAFEARRAY byte	POST body.
VT_BYREF ...	Con trỏ tới giá trị	Event parameters cho phép source/sink sửa.

VARIANT_TRUE có giá trị -1, không phải 1. Code so sánh với 0 thường vẫn hoạt động, nhưng khi dựng lại prototype phải giữ kiểu đúng để hiểu assignment và BYREF.

6.4 DOM access qua get_Document



```
docDisp->QueryInterface(IID_IHTMLDocument2, (void**)&doc);
```

Từ IHTMLDocument2, code có thể truy cập all/forms/body/cookie, tìm element theo id/name/tag, đọc value hoặc chèn script/HTML. Trong binary, chuỗi tên element, domain, form field và script fragment thường là IOC/behavior clue tốt hơn tên hàm.

6.5 Điều khiển trình duyệt: Navigate và put_Visible

```
browser->put_Visible(VARIANT_TRUE);
browser->Navigate(url, &flags, &target, &postData, &headers);
```

Trong sample dùng IID_IWebBrowser2, hai method thường được nhận diện qua vtable offset x86: put_Visible tại 0xA4 và Navigate tại 0x2C. Kết hợp này làm cửa sổ IE hiện ra rồi chuyển đến URL chỉ định. Nếu URL thuộc domain quảng cáo/C2, hành vi có thể là ad injection, redirect hoặc download.

6.6 Network hooking: phân biệt các lớp

Kỹ thuật	Vị trí	Khả năng
Browser events	DWebBrowserEvents2	Quan sát navigation lifecycle và tham số automation.
DOM manipulation	MSHTML interfaces	Đọc/sửa document sau khi render.
WinINET/URLMon hook	API/network stack	Quan sát request/response ở tầng thấp hơn.
Socket hook	Winsock	Chỉ thấy byte stream; TLS có thể đã mã hóa.

Không gọi mọi hoạt động BHO là network hooking. Nếu code chỉ dùng BeforeNavigate2 hoặc DOM, hãy mô tả chính xác là browser event interception/DOM access. Chỉ gọi API hook khi có patch IAT, inline hook, detour hoặc interception tương ứng.

6.7 Quy trình reverse hành vi từ Invoke

Bước	Câu hỏi
1. Map DISPID	Nhánh switch này tương ứng event nào?
2. Rebuild DISPPARAMS	cArgs bao nhiêu, rgvarg đảo thứ tự thế nào?
3. Decode VARIANT	vt là BSTR, DISPATCH, ARRAY hay BYREF?
4. Follow data	URL/POST/cookie được so sánh, ghi file hay gửi đi đâu?
5. Observe mutation	Code có sửa Cancel, URL, DOM hoặc headers không?
6. Confirm runtime	Breakpoint Invoke và ghi lại dispIdMember/VARIANT.

PHẦN 7. BẢO MẬT, PHÒNG VỆ VÀ SỰ THOÁI TRÀO

7.1 Vì sao BHO phù hợp với adware/spyware

BHO có persistence gắn với browser startup, thực thi trong process được người dùng tin tưởng, truy cập browser automation/DOM và nhận event đúng thời điểm. Network activity có thể xuất hiện dưới iexplore.exe thay vì một process lạ. Với người dùng thời IE, add-on thường khó nhận biết hơn ứng dụng độc lập.

7.2 Man-in-the-browser

MitB hoạt động tại endpoint. Dữ liệu form tồn tại ở dạng rõ trong DOM/VARIANT trước khi được mã hóa để gửi. Do đó HTTPS vẫn có thể hợp lệ trong khi extension độc hại đọc hoặc thay nội dung. Phòng vệ phải kiểm soát integrity của endpoint và extension, không chỉ network encryption.

7.3 Protected Mode và integrity level

Protected Mode đưa tab IE vào low integrity để hạn chế ghi vào vùng medium/high integrity. Điều này giảm tác động nhưng không loại bỏ rủi ro đọc/sửa nội dung trong chính browser context. Broker processes và policy làm luồng thực thi phức tạp hơn; analyst cần ghi nhận process, integrity level và bitness khi debug.

7.4 32-bit và 64-bit

Tình huống	Hệ quả
BHO x86, IE process x86	Có thể nạp nếu registry/policy phù hợp.
BHO x86, IE process x64	Không thể LoadLibrary trực tiếp do khác kiến trúc.
Registry 32-bit view	Có thể nằm dưới WOW6432Node hoặc được redirect.
regsvr32	System32 chứa bản 64-bit; SysWOW64 chứa bản 32-bit.

7.5 Dấu hiệu giám sát và hunting

- Registry creation dưới CLSID/InprocServer32 và Browser Helper Objects.
- regsvr32 tạo bởi process bất thường hoặc đăng ký DLL từ AppData/Temp/Downloads.
- iexplore.exe nạp DLL unsigned từ user-writable directory.
- Browser process tạo file, process con hoặc network connection bất thường.
- CLSID hợp pháp bị trỏ sang DLL khác: dấu hiệu COM hijacking.
- Sai lệch giữa bitness/path, timestamp, signer, product name và install source.

7.6 COM hijacking và CLSID reuse

Một CLSID không phải mật mã hay danh tính bảo mật. Malware có thể tái sử dụng CLSID từ project mẫu hoặc chiếm một CLSID đã được host kích hoạt. Vì vậy danh sách IOC CLSID hữu ích cho hunting nhưng phải kết hợp đường dẫn InprocServer32, hash, signer, creation time và host process.

Case CLSID 3543619C-D563-43F7-95EA-4DA7E1CC396A

CLSID này từng xuất hiện trong project mẫu BHO và nhiều họ adware/malware tái sử dụng. Nó nhận diện class registration, không chứng minh interface. Trong reverse, IID D30C1661-CDAF-11D0-8A3E-00C04FC9E26E mới là bằng chứng cho IWebBrowser2.

7.7 Vì sao BHO suy tàn

Internet Explorer đã bị thay thế bởi Microsoft Edge dựa trên Chromium. Mô hình extension hiện đại dùng manifest, permission boundaries, isolated worlds và IPC thay vì nạp DLL COM tùy ý vào renderer/browser theo kiểu BHO. Tuy vậy, BHO vẫn quan trọng trong malware analysis vì tồn tại trên hệ thống legacy, trong mẫu lịch sử và trong các kỹ thuật COM persistence/hijacking liên quan.

PHẦN 8. PHỤ LỤC KỸ THUẬT VÀ CHEAT SHEET

8.1 GUID chuẩn thường gặp

Tên	GUID	Vai trò
IID_IUnknown	00000000-0000-0000-C000-000000000046	Interface gốc COM.
IID_IClassFactory	00000001-0000-0000-C000-000000000046	Tạo object COM.
IID_IObjectWithSite	FC4801A3-2BA9-11CF-A229-00AA003D7352	Nhận/trả site pointer.
IID_IWebBrowser2	D30C1661-CDAF-11D0-8A3E-00C04FC9E26E	Browser automation.
IID_IConnectionPointContainer	B196B284-BAB4-101A-B69C-00AA00341D07	Tìm connection point.
IID_IConnectionPoint	B196B286-BAB4-101A-B69C-00AA00341D07	Advise/Unadvise sink.
DIID_DWebBrowserEvents2	34A715A0-6587-11D0-924A-0020AFC7AC4D	IE outgoing events.

8.2 GUID endian conversion

Bytes trong file:
61 16 0C D3 AF CD D0 11 8A 3E 00 C0 4F C9 E2 6E

Tách trường:
Data1 = D30C1661
Data2 = CDAF
Data3 = 11D0
Data4 = 8A 3E 00 C0 4F C9 E2 6E

GUID = D30C1661-CDAF-11D0-8A3E-00C04FC9E26E

Data1, Data2 và Data3 được lưu little-endian trong binary; tám byte Data4 giữ nguyên thứ tự. IDA có thể tự áp kiểu GUID/IID nếu type library hoặc signature phù hợp.

8.3 IObjectWithSite vtable x86

Slot	Offset	Method	Prototype rút gọn
0	0x00	QueryInterface	(REFIID, void **ppv)
1	0x04	AddRef	()
2	0x08	Release	()
3	0x0C	SetSite	(IUnknown *site)
4	0x10	GetSite	(REFIID, void **site)

8.4 IConnectionPointContainer vtable x86

Slot	Offset	Method
0	0x00	QueryInterface
1	0x04	AddRef
2	0x08	Release
3	0x0C	EnumConnectionPoints
4	0x10	FindConnectionPoint

8.5 IConnectionPoint vtable x86

Slot	Offset	Method
0	0x00	QueryInterface
1	0x04	AddRef
2	0x08	Release
3	0x0C	GetConnectionInterface
4	0x10	GetConnectionPointContainer
5	0x14	Advise
6	0x18	Unadvise
7	0x1C	EnumConnections

8.6 IDispatch vtable x86

Slot	Offset	Method
0	0x00	QueryInterface
1	0x04	AddRef
2	0x08	Release
3	0x0C	GetTypeInfoCount
4	0x10	GetTypeInfo
5	0x14	GetIDsOfNames
6	0x18	Invoke

8.7 IWebBrowser2 offsets quan trọng x86

Offset	Method	Reverse clue
0x1C	GoBack	Không tham số ngoài this.
0x20	GoForward	Không tham số ngoài this.
0x24	GoHome	Không tham số ngoài this.
0x28	GoSearch	Không tham số ngoài this.
0x2C	Navigate	URL BSTR + bốn VARIANT*.
0x30	Refresh	Không tham số ngoài this.
0x38	Stop	Dừng navigation/download.
0x48	get_Document	Output IDispatch**.
0x7C	get_Busy	Output VARIANT_BOOL*.
0xA0	get_Visible	Output VARIANT_BOOL*.
0xA4	put_Visible	Input VARIANT_BOOL; -1 thường là TRUE.

8.8 DISPIDs tiêu biểu

Decimal	Hex	Tên	Ghi chú
250	0xFA	BeforeNavigate2	Trước navigation; có Cancel.
251	0xFB	NewWindow2	Tạo window mới.
252	0xFC	NavigateComplete2	Navigation hoàn tất.
253	0xFD	OnQuit	Browser sắp thoát.
259	0x103	DocumentComplete	Document đã hoàn tất.
263	0x107	WindowClosing	Đóng window.
273	0x111	NewWindow3	New window với URL/context.

PHẦN 9. QUY TRÌNH REVERSE BHO THỰC CHIẾN

9.1 Mục tiêu đầu ra

Một báo cáo tốt không chỉ nói 'đây là BHO'. Nó phải chứng minh class registration, interface được sử dụng, event được đăng ký, callback xử lý event nào, dữ liệu nào bị đọc/sửa và tác động quan sát được.

Câu hỏi	Bằng chứng tối thiểu
DLL được nạp thế nào?	Registry CLSID/InprocServer32/BHO key hoặc code DllRegisterServer.
Interface nào?	IID tại QueryInterface hoặc prototype/type library tương đương.
Method nào?	Vtable offset khớp layout và signature.
Event nào?	DISPID trong Invoke và prototype DWebBrowserEvents2.
Dữ liệu nào?	VARIANT decoding, strings, DOM/API calls, runtime values.
Hành vi cuối?	URL/file/registry/network/process observable.

9.2 Workflow trong IDA/Ghidra

1. Triage PE: bitness, exports, imports, resources, strings, signer và compiler.
2. Tìm DllGetClassObject/DllRegisterServer và các CLSID được so sánh hoặc ghi Registry.
3. Định dạng các constant 16 byte thành GUID; đặt tên IID_/DIID_.
4. Nhận diện QueryInterface từ slot 0 và prototype; đặt type cho output pointer.
5. Dựng struct object từ các field chứa site, browser, connection point và cookie.
6. Tra vtable để rename indirect calls; chú ý decimal/hex và x86/x64 pointer size.
7. Tìm IDispatch::Invoke, map DISPID và dựng DISPPARAMS/VARIANT.
8. Theo dữ liệu đến sink cuối: file, Registry, DOM mutation, URL hoặc network API.
9. Debug trong VM: breakpoint SetSite, QueryInterface, Advise và Invoke; ghi runtime types.
10. Viết kết luận theo chuỗi bằng chứng, không dựa vào tên malware hoặc CLSID đơn lẻ.

9.3 Mẫu phân tích lời gọi gián tiếp

```
mov eax, [esi+10h] ; browser interface pointer
mov ecx, [eax] ; vtable
push ... ; arguments
call dword ptr [ecx+2Ch]
```

```
Known type: IWebBrowser2*
Offset: 0x2C
Result: IWebBrowser2::Navigate
```

Nếu chưa có known type, offset 0x2C không đủ. Hãy quay lại nơi field +0x10 được gán. Nếu nó là output của QueryInterface với IID_IWebBrowser2 thì type propagation hoàn tất.

9.4 Debug checklist

Breakpoint	Quan sát
DllGetClassObject	CLSID host yêu cầu và IID factory.
IObjectWithSite::SetSite	pUnkSite, object this, teardown NULL.
IUnknown::QueryInterface	riid và output pointer sau return.
IConnectionPoint::Advise	sink pointer và cookie.
IDispatch::Invoke	DISPID, cArgs, rgvarg, vt values.
IWebBrowser2::Navigate	URL, headers, post data.
RegSetValueEx	Registration/persistence values.

9.5 Các bẫy thường gặp

Bẫy	Cách tránh
Nhầm CLSID với IID	Hỏi: GUID này dùng để tạo class hay truyền vào QueryInterface?
Đọc 16 thành 0x16	Decompiler có thể hiện decimal; đổi 16 -> 0x10, 20 -> 0x14.
Bỏ method kế thừa	Vtable IWebBrowser2 bắt đầu từ IUnknown/IDispatch/IWebBrowser/...
Đọc DISPPARAMS xuôi	Automation lưu tham số theo thứ tự đảo ngược.
Tin prototype decompiler	Khôi phục calling convention và số tham số từ interface definition.
Coi SetSite là payload	SetSite thường chỉ wiring; logic thật ở Invoke/worker.
Coi mọi GUID match là tuyệt đối	Malware có thể reuse CLSID; dùng call-site semantics để xác nhận.

9.6 Case walkthrough rút gọn

```

riid = IID_IWebBrowser2
site->QueryInterface(riid, &this->browser)

this->browser->QueryInterface(IID_IConnectionPointContainer, &cpc)
cpc->FindConnectionPoint(DIID_DWebBrowserEvents2, &this->cp)
this->cp->Advise(this->eventSink, &this->cookie)

// elsewhere
this->browser->put_Visible(VARIANT_TRUE)
this->browser->Navigate(url, ...)

```

Chuỗi trên trả lời ba nhóm câu hỏi: interface là IWebBrowser2; subscription dùng FindConnectionPoint và Advise; hành vi điều khiển browser dùng put_Visible và Navigate. Cần đọc đúng phạm vi câu hỏi để không trả hai method wiring thay cho hai method được gọi từ IWebBrowser2.

THUẬT NGỮ

Thuật ngữ	Định nghĩa ngắn
Apartment	Mô hình đồng bộ/threading của COM object và proxy.
BHO	DLL COM extension được IE/Explorer nạp và gắn site.
BSTR	Chuỗi Automation có length prefix, thường UTF-16.
CLSID	GUID nhận diện COM class.
Connection point	Object quản lý subscription cho outgoing interface.
Cookie	Token DWORD nhận từ Advise và dùng với Unadvise.
DIID	IID dành cho dispatch/event interface.
DISPID	Số nhận diện member/event khi gọi qua IDispatch.
Event sink	COM object nhận callback từ source.
HRESULT	Mã trạng thái 32-bit; giá trị âm biểu thị failure.
IID	GUID nhận diện COM interface.
InprocServer32	Registry mapping CLSID tới DLL COM server.
Site	Host-side object được truyền cho extension qua SetSite.
VARIANT	Tagged union dùng bởi COM Automation.
Vtable	Bảng function pointer hiện thực interface.

TÀI LIỆU THAM KHẢO

- Michael Sikorski, Andrew Honig. Practical Malware Analysis. No Starch Press, 2012.
- Microsoft Learn. IUnknown::QueryInterface, IObjectWithSite, IConnectionPointContainer, IConnectionPoint và IDispatch documentation.
- Microsoft Learn. IWebBrowser2 interface và Internet Explorer WebBrowser control reference.
- Microsoft Learn. DWebBrowserEvents2 event interface, BeforeNavigate2 và DocumentComplete reference.
- Microsoft Windows SDK headers: unknwn.h, ocidl.h, exdisp.h, exdispid.h, servprov.h.
- Microsoft documentation về COM in-process servers, class factories, reference counting và apartments.
- MITRE ATT&CK; T1546.015 - Event Triggered Execution: Component Object Model Hijacking.

Kết luận

Reverse BHO hiệu quả không bắt đầu từ việc đọc toàn bộ DLL. Hãy dựng chuỗi object: registry/CLSID → class factory → IObjectWithSite::SetSite → IID/interface pointer → connection point → IDispatch::Invoke → dữ liệu/hành vi. Khi type đã đúng, phần lớn lời gọi gián tiếp sẽ trở thành các method có tên và mục đích rõ ràng.